

## Friction and Rolling in 2D

For the exercises that follow, implement the code into files called `rigidbody2d_friction.py` and `rigidbody2d_rolling.py` and hand them in on avenue. For the written answers and discussion, hand-in a separate pdf file with your answers.

**Exercise 1: Complete your `rigidbody2d_friction.py` code as discussed and worked on in class to include friction.** (see VG3\_Friction.pdf lecture notes). You should have begun with `rigidbody2d_friction0.py` from Avenue or similar. You also need to get `rigidbody2d_mod.py` and `rigidbody_collision_mod.py` as function are imported from them. You should use the standard versions of these provided.

Note that for any test problem, if you change the mass or the radius you will need to update the collision radius, inverse mass and the inverse moment of inertia as well.

### Exercises 2: Friction on an inclined plane

Set up an inclined plane test case. Note that the most important things here are the angle (in radians) and that the bottom body has the inverse mass and inverse moment of inertia set to 0 so it does not move. For this test use an angle of 13 degrees (or 0.227 radians) and friction coefficients around 0.5. You probably want to use low values of the coefficient of restitution (e.g.  $e = 0.1$ ).

**(a) Our code is not perfect, there can be small rocking and sliding motions even for strong friction. Explore the working code. Describe how/why you think this is happening. You may wish to look at the frame to frame behaviour of the square to understand this.**

For a given angle, there is a minimum value of the coefficient of static friction equal to  $\tan(\text{angle})$  required to prevent acceleration via friction. Given that our code is not perfect, there will be small motions (constant speed). However, if the friction is too low, it should actually accelerate not just move at constant speed.

**(b) Explore this scenario and vary the friction coefficient (use the same value for static and kinetic friction each time). See how well the engine model reproduces the expected behaviour. Describe what happens and comment on the behaviour of the engines.**

**(c) Try out your own ideas to see if you can stop the rocking or sliding or improve the performance on the critical angle test for (b). You can refer to the lecture notes or come up with your own ideas. Explain the strategy you came up with and how it works. Save your new version as `rigidbody2d_friction_extra.py` to hand-in.**

**Exercise 3:** We worked on `rigidbody2d_rolling.py` in class. **Complete the code as described in the lecture notes** (see VG3\_Rolling.pdf). The code you hand in should perform a true cylinder of radius 1.0 rolling down an incline of 13 degrees (similar to exercise 2) as the default scenario.

```
#cylinder
self.RigidBody2Ds[0].radius = 1.0
self.RigidBody2Ds[0].collisionRadius = 1.1*self.RigidBody2Ds[0].radius
self.RigidBody2Ds[0].setScale(1.0)
self.RigidBody2Ds[0].setPos(-2.,3,self.RigidBody2Ds[0].radius-0.4)
self.RigidBody2Ds[0].doGravity = True
self.RigidBody2Ds[0].shape = "cylinder"
```

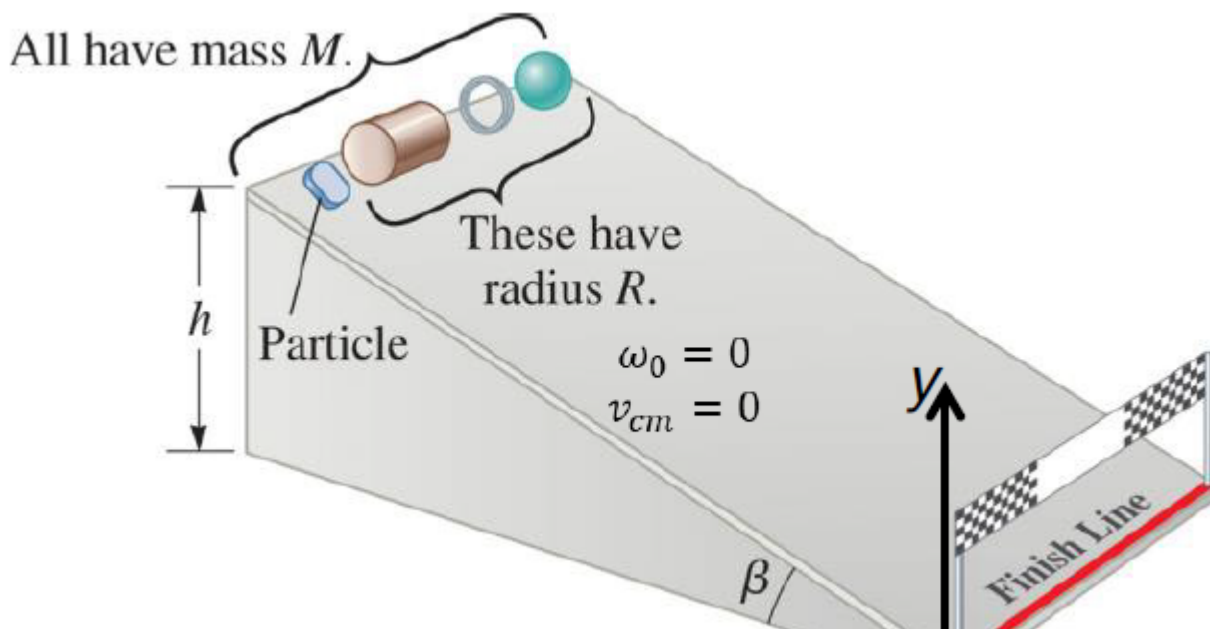
```

self.RigidBody2Ds[0].inverseMomentOfInertia =
self.RigidBody2Ds[0].inverseMass/(self.RigidBody2Ds[0].radius**2)
self.tube.instanceTo(self.RigidBody2Ds[0])

self.RigidBody2Ds[1].radius = 4.0
self.RigidBody2Ds[1].collisionRadius = 1.8*self.RigidBody2Ds[1].radius
self.RigidBody2Ds[1].setScale(4.0)
self.RigidBody2Ds[1].setPos(0.,3,-self.RigidBody2Ds[1].radius-1.)
self.RigidBody2Ds[1].inverseMass = 0.0
self.RigidBody2Ds[1].inverseMomentOfInertia = 0.0
self.RigidBody2Ds[1].theta = 13*pi/180
self.cube.instanceTo(self.RigidBody2Ds[1])

```

#### Exercise 4: The Race!



Perform the race described in the lecture notes using the above set-up with an angle of 13 degrees.

The 4 racers are a hollow cylinder ( $I = MR^2$ ), solid cylinder ( $I = \frac{1}{2} MR^2$ ), sphere ( $I = \frac{2}{5} MR^2$ ) and sliding particle (use a square with no friction,  $I$  should not matter, e.g.  $I^{-1} = 0$ ). Modify your code to have 4 possible scenarios corresponding to the 4 objects above.

**Measure the final speed (e.g. at the end of the block) in the same way for all 4 cases and report your answers. Do these results match expectations? Do some measurements if necessary to determine what is going on and try to explain why it did or did not work.**